

Constrained Zoned-Object Architecture (CZOA): A Unified Framework for Building Secure and Intelligent Integrated Organizational Systems

Harris Wang

School of Computing and Information Systems

Athabasca University

Athabasca, Canada

harrisw@athabascau.ca

Abstract

Modern enterprise systems must be both secure and adaptive—able to enforce strict access policies while responding intelligently to changing conditions. Yet the theories that explain intelligent behaviour and the engineering methodologies that build reliable systems have evolved separately, leaving a gap that forces designers to choose between formal guarantees and adaptive intelligence. This paper introduces the Constrained Zoned-Object Architecture (CZOA), a unified formalism that bridges this gap. CZOA integrates the mathematical rigour of Constrained Object Hierarchies (COH) with the practical, validated structure of the Zoned Role-Based (ZRB) framework. We show that enterprise systems are a natural species of intelligent systems, and that the combined framework preserves minimality while enabling new capabilities: neural-enhanced permission mining, semantic embeddings for cross-zone understanding, adaptive access control, and continuous daemon-based enforcement. A complete Python implementation (CZOI toolkit) and a logic language (UniLang) make the formalism accessible to engineers. Five case studies across healthcare, finance, traffic, education, and supply chain demonstrate that CZOA delivers sub-millisecond permission checks, up to 23% improvement in operational metrics, and high-accuracy anomaly detection (94% precision) while maintaining rigorous security constraints. The paper concludes with a discussion of limitations, threats to validity, and future research directions.

ACM Classification: I.2.11 [Artificial Intelligence]: Distributed Artificial Intelligence -- Intelligent agents; D.2.11 [Software Engineering]: Software Architectures -- Domain-specific architectures; K.6.5 [Management of Computing and Information Systems]: Security and Protection

Keywords: intelligent enterprise systems; access control; adaptive systems; formal methods; constrained object hierarchies; role-based architecture; integrated organizational systems; system of systems

1. Introduction

Consider a regional hospital network during a flu outbreak. Patient volumes triple overnight. Staff must be redeployed, permissions must be temporarily elevated, and new clinical protocols must be enforced—all while maintaining strict compliance with privacy regulations and preventing security breaches. Today's

enterprise systems cannot handle such a surge without days of manual reconfiguration. Conversely, a purely "intelligent" system that learns to adapt might grant access in ways that violate safety constraints or regulatory rules.

This scenario illustrates a fundamental fragmentation in computing: theories of intelligence and methodologies for building practical, secure systems remain largely disconnected. On one side, research in artificial general intelligence (AGI) produces elegant mathematical frameworks that explain how intelligent behaviour emerges from hierarchical composition, constraint satisfaction, and adaptive learning. These frameworks offer profound insights but often lack direct pathways to implementation in operational systems.

On the other side, enterprise system engineering has developed sophisticated methodologies for building complex, secure, and maintainable systems that mirror organisational structures. These methodologies provide formal definitions, permission calculi, and validated lifecycle processes, yet they do not explicitly address how such systems might exhibit adaptive intelligence or learn from experience.

The cost of this fragmentation is high. Intelligent systems built without sound engineering foundations become unmaintainable [Souppaya et al. 2022]; enterprise systems built without intelligent adaptation cannot respond to dynamic environments [Hendrycks et al. 2022]. What is needed is a unified formalism that inherits theoretical rigour and practical proven effectiveness, along with a concrete implementation framework that makes the formalism accessible to software engineers.

The insight. This paper's central insight is that **enterprise systems are a species of intelligent systems**. An organisation—with its hierarchical departments, functional roles, operational procedures, data, policies, and adaptation mechanisms—instantiates the same structural patterns used to model general intelligence. The zone tree of an organisational hierarchy is a hierarchical decomposition; roles and their permissions correspond to methods and attributes; inheritance mappings implement constraint-regulated behaviour; and the system's ability to respond to organisational changes constitutes a form of adaptation.

Two foundational frameworks. This work builds upon two previously developed frameworks. The Constrained Object Hierarchies (COH) framework [Wang 2025] models intelligent systems as a 9-tuple of components, attributes, methods, neural components, embeddings, identity constraints, trigger constraints, goal constraints, and constraint daemons. COH provides a mathematically rigorous foundation for understanding how intelligent behaviour emerges from hierarchical composition and constraint satisfaction.

Concurrently, the Zoned Role-Based (ZRB) framework [Wang 2026] offers a formally validated methodology for designing, implementing, and maintaining complex enterprise systems with built-in access control and organisational alignment. ZRB introduces zones as organisational units, roles with base permissions, intra-zone role hierarchies, inter-zone role mappings (γ), and a comprehensive constraint model.

The present work unifies these frameworks into a single coherent formalism—Constrained Zoned-Object Architecture (CZOA)—that inherits the theoretical depth of COH and the engineering pragmatism of ZRB. Critically, we extend the formalism recursively: each zone is itself a full CZOA system, enabling a **native**

system-of-systems architecture where every organisational unit retains local autonomy while remaining safely governed by global enterprise constraints.

Contributions. This paper makes the following contributions:

1. **A unified formalism (CZOA)** integrating the COH 9-tuple with ZRB's zone-role structure, preserved minimality and orthogonality, and extended with **recursive subsystem definitions**.
2. **Formal mapping theorems** demonstrating that well-formed ZRB specifications embed into recursive CZOA and vice versa.
3. **Category-theoretic foundations** showing that recursive CZOA objects form a Cartesian closed category, enabling compositional reasoning about systems of systems.
4. **An extended permission calculus** incorporating neural components, embeddings, and recursive parent-override semantics.
5. **A constraint daemon architecture** generalising ZRB's constraint model with hierarchical, continuous monitoring processes.
6. **UniLang**: a concrete logic language for formal constraint specification, with a parser and modular inference engine.
7. **The CZOI toolkit**: a complete Python implementation providing modules for zones (as subsystems), roles, permissions, constraints, neural components, embeddings, daemons, simulation, and web framework integrations.
8. **Five condensed case studies** with quantitative evaluation across healthcare, finance, traffic management, higher education, and supply chain.

Paper organisation. Section 2 briefly reviews the two foundational frameworks, emphasising their integration points. Section 3 presents the unified, recursive CZOA formalism. Section 4 outlines category-theoretic foundations in a compact form. Section 5 develops the learning-enhanced permission calculus. Section 6 describes the CZOI toolkit and UniLang. Section 7 evaluates CZOA through five case studies, including baseline methodology and statistical significance. Section 8 discusses limitations, threats to validity, and future work. Section 9 concludes. A glossary of acronyms and key terms is provided at the end.

2. Foundational Frameworks

2.1 Constrained Object Hierarchies (COH)

COH [Wang 2025] models an intelligent system as a 9-tuple:

$$O = (C, A, M, N, E, I, T, G, D)$$

- **C (Components)**: Hierarchical decomposition into sub-objects (DAG).

- **A (Attributes):** State variables.
- **M (Methods):** Executable transformations.
- **N (Neural Components):** Parameterised functions for learning.
- **E (Embedding):** Semantic mapping to a Hilbert space.
- **I (Identity Constraints):** Invariants for all reachable states.
- **T (Trigger Constraints):** Event-condition-action rules.
- **G (Goal Constraints):** Optimisation objectives.
- **D (Constraint Daemons):** Continuous monitoring processes.

COH is proven sound, complete, minimally expressive, and orthogonal.

2.2 Zoned Role-Based Framework (ZRB)

ZRB [Wang 2026] provides a formal methodology for enterprise systems. Its core entities are:

- **Users (U):** Authenticated identities.
- **Applications (A) and Operations (Op):** Applications expose operations.
- **Roles (R):** Job functions with base permissions.
- **Zones (Z):** Organisational units recursively defined as $z = (Z_{\text{child}}(z), R_z, A_z, U_z)$, where $Z_{\text{child}}(z)$ are child zones, R_z are roles defined in z , A_z are applications provisioned in z , and U_z are users affiliated with z .

Zones form a rooted tree called the **Zone Tree** with the containment principle: $U_z \subseteq U_{\text{parent}(z)}$. This ensures that users in a child zone are automatically in the parent zone.

Within each zone, roles are organised in an **intra-zone role hierarchy**: a partial order $\succeq_z$ where $r_1 \succeq_z r_2$ means that r_1 inherits the permissions of r_2 . This enables senior roles to have all permissions of junior roles.

Between zones, **inter-zone role mappings** $\gamma: (z_{\text{child}}, r_{\text{child}}) \mapsto (z_{\text{parent}}, r_{\text{parent}})$ with weights and priorities allow permissions to flow across zone boundaries. The weight (0 to 1) represents the degree of permission transfer (e.g., requiring additional training or approval), and priority resolves conflicts when multiple mappings apply.

The **effective permissions** of a role r in zone z are computed as:

$$P_{\text{effective}}(r, z) = P_{\text{base}}(r) \cup \bigcup_{r_j: r \succeq_z r_j} P_{\text{base}}(r_j) \cup \bigcup_{(z', r') \in \gamma\text{-ancestry}(r, z)} P_{\text{base}}(r')$$

Constraints include separation of duty (SoD), temporal, attribute, and context restrictions. ZRB has been validated in production, showing 65–83% reduction in design time and 3–4× reduction in implementation effort.

Note: While ZRB defines zones as 4-tuples, the recursive CZOA defined in Section 3 elevates each zone to a full 10-tuple system, generalising ZRB's structure while preserving its containment principles.

2.3 Integration Points

The frameworks share deep structural correspondences, summarised in Table 1. This isomorphism suggests that ZRB is a specialisation of COH to the enterprise domain, where components are zones, methods are operations, constraints are security policies, and learning is made explicit via neural components.

Table 1: Structural correspondences between COH and ZRB

COH Component	ZRB Counterpart	Integration Insight
Components (C)	Zone Tree (Z)	Both form hierarchical DAGs with containment
Attributes (A)	Application data, user attributes	State variables
Methods (M)	Operations (Op)	Executable units
Neural (N)	(Implicit)	Learning for dynamic permission adaptation
Embedding (E)	Zone/role context	Semantic representation
Identity (I)	Zone containment, role definitions	Invariants
Trigger (T)	Event-condition-action constraints	Reactive behaviour
Goal (G)	Business objectives, KPIs	Optimisation

COH Component	ZRB Counterpart	Integration Insight
Daemons (D)	Continuous monitoring, audit	Real-time enforcement

3. The CZOA Formalism

Definition 1 (Recursive Constrained Zoned-Object Architecture). A CZOA system is a 10-tuple:

$$\mathcal{S} = (Z, R, U, A, O, N, E, \Gamma, \Phi, \Delta)$$

where the components are defined recursively as follows:

1. **Z (Zones – Subsystems):** A finite set of **subsystems** forming a rooted tree $T = (Z, E)$ with root z_{root} . Crucially, **every zone $z \in Z$ is itself a valid CZOA system**:

$$z = (Z_z, R_z, U_z, A_z, O_z, N_z, E_z, \Gamma_z, \Phi_z, \Delta_z)$$

where $Z_z \subseteq Z$ denotes the child subsystems of z . The recursion terminates when $Z_z = \emptyset$ (a leaf subsystem).

Interpretation: Every organisational unit—whether a top-level enterprise, a regional division, a hospital, or an emergency department—is a fully autonomous system with its own roles, operations, neural components, constraints, and daemons.

2. **R (Roles):** A finite set of roles global to $\mathcal{S}$. Each role $r \in R$ has a zone of definition $\text{zone}(r) \in Z$, base permissions $P_{\text{base}}(r) \subseteq O$, and an intra-zone seniority relation $\succeq_z$. Roles are inherited by child subsystems (i.e., $R_{z_{\text{child}}} \subseteq R_{z_{\text{parent}}}$).
3. **U (Users):** Finite set of users, each with credentials and attributes.
4. **A (Applications):** A finite set of **structural modules** or **service boundaries**. Each application $a \in A$ is defined by the set of executable actions it exposes, denoted $O_a = \{a.o_1, \dots, a.o_n\}$. Applications serve as the **administrative and semantic namespace** for operations, but they are **not** the direct target of permission assignments. That is, a user is never granted "access to an application" in the abstract; instead, the system grants rights over the specific atomic actions that the application provides.

Design rationale. Separating the module (A) from the action (O) preserves orthogonality: applications provide deployability, versioning, and user-interface grouping, while operations provide the granular behavioural units needed for fine-grained security and auditing. This distinction also mirrors modern service-oriented and microservice architectures, where a service (application) exposes a well-defined API (set of operations).

5. **O (Operations):** A finite set of **atomic executable units** that represent the actual functional capabilities of the system. The global operation set is defined by the union over all applications:

$$O = \bigcup_{a \in A} O_a$$

with the convention that each operation is uniquely identified by its parent application, typically written as `application.operation` (e.g., `GCPay.process_payroll`).

Crucially, the permission calculus Φ and the effective permission set $P_{\text{effective}}(r, z)$ are defined *only* over O . For any role r , its base permissions satisfy:

$$P_{\text{base}}(r) \subseteq O$$

This means that a role is granted a set of *atomic actions*, and the application grouping is used solely for organisation, constraint scoping, and embedding similarity. The containment principle for recursive subsystems (Section 3) likewise operates on the operation level: if a child zone inherits a role from its parent, it inherits the parent's set of operations O_{parent} , but the child may define additional operations O_{child} that are not exposed to the parent, preserving local autonomy.

6. **N (Neural Components):** Parameterised functions enabling learning and adaptation, defined locally per subsystem:

$$N_z = \{f_\theta: X \rightarrow Y \mid \theta \in \Theta\} \text{ for each } z \in Z$$

and

globally

$$N = \bigcup_{z \in Z} N_z \cup N_{\text{root}}$$

Interpretation: Child subsystems can deploy highly specialised ML models (e.g., sepsis LSTM in Emergency), while the root can run cross-subsystem anomaly detectors.

7. **E (Embedding):** Semantic mapping functions, now **layered**:

- **Local embeddings:** $E_z: (Z_z \cup R_z \cup U_z \cup A_z \cup O_z) \rightarrow \mathcal{H}_z$ for each subsystem z .
- **Global alignment functor:** $E_{\text{align}}: \mathcal{H}_z \rightarrow \mathcal{H}_{\text{root}}$ that projects local embeddings into a shared inter-system Hilbert space $\mathcal{H}_{\text{root}}$ for cross-subsystem similarity reasoning.
Interpretation: Local embeddings capture domain-specific semantics; the root embedding enables safe gamma mappings between roles in different subsystems.

8. **Γ (Constraint System):** A 4-tuple $\Gamma = (I, T, G, C)$ defined **hierarchically**:

- **Identity (I):** Invariants, including recursive containment: $\forall z_{\text{child}} \in Z_z, U_{z_{\text{child}}} \subseteq U_z$ and $\Gamma_{z_{\text{child}}} \subseteq \Gamma_z^*$ (child constraints must not violate parent constraints).
- **Trigger (T):** Event-condition-action rules.
- **Goal (G):** Optimisation objectives (KPIs) for each subsystem.
- **Access (C):** SoD, temporal, attribute, context constraints.
Interpretation: Parent constraints override child constraints. A child may add *stricter* local rules, but cannot weaken a parent rule.

9. **Φ (Permission Calculus):** A two-stage recursive decision function:

- **Local check:** Φ_z evaluates the request using local roles R_z , local constraints Γ_z , and local neural outputs N_z .
- **Parent override:** If the operation involves cross-subsystem resources or the local decision is inconclusive, Φ_{parent} is invoked. The effective permissions are:

$$P_{\text{effective}}(r, z) = P_{\text{base}}^z(r) \cup \bigcup_{r' \in \text{seniority}_z(r)} P_{\text{base}}^z(r') \cup \bigcup_{z_{\text{child}} \in Z_z} \gamma(z_{\text{child}}, r) \cup \Phi_{\text{parent}}^{-1}(r, z)$$

where γ denotes inter-subsystem role mappings with weights and priorities.

10. **Δ (Daemons):** Continuous monitoring processes, also **recursive**:

$$\Delta_z = \{d_k: \mathcal{S}_z^T \rightarrow \mathcal{A}\}_{k=1}^{r_z}$$

for each subsystem z , and global daemons Δ_{root} that monitor cross-subsystem flows. Daemons form a monitoring hierarchy; child daemons can publish signals (e.g., STATE_CRITICAL) to parent daemons for coordinated responses.

Interpretation Note (The "Object" in CZOA). Although the 10-tuple $\mathcal{S}$ does not contain a top-level atom named "Object", the term refers to **every zone** $z \in Z$. In the recursive formulation, each zone is a self-contained entity that encapsulates identity (U_z), state (A_z), behaviour (O_z), and invariants (Γ_z)—the hallmarks of an object in the object-oriented paradigm. Thus, the architecture is fundamentally composed of *zoned objects* that are *constrained* by a global policy system. This naming also preserves the direct lineage to the Constrained Object Hierarchies (COH) framework upon which CZOA is built.

Definition 2 (Recursive CZOA State). A state $s \in \mathcal{S}$ is a tuple:

$$s = (u_{\text{active}}, z_{\text{current}}, \text{ancestry}(z_{\text{current}}), \rho, \sigma, \tau)$$

where $u_{\text{active}} \in U$ is the current user, $z_{\text{current}} \in Z$ is the current subsystem context, $\text{ancestry}(z_{\text{current}})$ is the path from root to z_{current} (enabling permission inheritance lookup), $\rho: R \rightarrow [0,1]$ is role activation levels, σ maps attributes to values, and $\tau \in \mathcal{T}$ is current time.

Definition 3 (Recursive Transition System). A labelled transition $s \xrightarrow{o} s'$ is allowed iff the requested atomic operation $o \in O$ satisfies the local decision function:

$$\text{decide}_{\text{local}}(u_{\text{active}}, o, z_{\text{current}}, \rho, \sigma, \tau) = \text{ALLOW}$$

and (if the operation touches resources belonging to a parent zone, or if the local policy delegates the decision upward):

$$\text{decide}_{\text{parent}}(u_{\text{active}}, o, \text{parent}(z_{\text{current}}), \rho, \sigma, \tau) = \text{ALLOW}$$

Here, the decision function never refers to an application a as a whole; it only evaluates the specific operation o . This ensures that access control is **maximally granular** and that the recursive parent-override semantics (Theorem 1) operate on the operation level, preserving the principle of least privilege.

Remark 1 (Namespace vs. Permission Target). Although the 10-tuple defines both A and O , they are not peers in the permission calculus. The application set A induces a *partition* of O :

$$O = \sqcup_{a \in A} O_a$$

but no permission check ever inspects A directly. This design is essential for the **minimality** claim of CZOA: the ten components are orthogonal. If applications themselves were permission targets, we would need an extra layer of indirection and the framework would lose its formal elegance. Instead, applications are first-class entities for deployment, configuration, and semantic embedding, whereas operations are first-class entities for security enforcement.

Theorem 1 (ZRB Embedding into Recursive CZOA). Every well-formed ZRB specification $Z = (Z, R, U, A, O, \Gamma, \Phi)$ —where zones are 4-tuples $z = (Z_{\text{child}}, R_z, A_z, U_z)$ —can be embedded in a recursive CZOA system $\mathcal{S}$ by constructing, for each ZRB zone z_{ZRB} , a CZOA subsystem:

$$z_{\text{CZOA}} = (Z_{\text{child}}, R_z, U_z, A_z, O_z, N_z = \{\text{id}\}, E_z = E_{\text{tree}}, \Gamma_z = (I_{\text{contain}}, \emptyset, G_{\text{opt}}, C_z), \Phi_z = \Phi_{\text{ZRB}}, \Delta_z = \emptyset)$$

where N_z is the identity function, E_{tree} is the trivial tree-based embedding, and Δ_z is empty. The ZRB containment $U_{z_{\text{child}}} \subseteq U_{z_{\text{parent}}}$ is preserved as an identity constraint in Γ_z . Thus CZOA conservatively extends ZRB with full subsystem autonomy.

Proof sketch (expanded). In the embedding construction, each ZRB application a_{ZRB} is mapped to a CZOA application a_{CZOA} with the same name. The ZRB operation set O_{ZRB} becomes the CZOA global operation set O_{CZOA} , and the ZRB permission assignment $P_{\text{ZRB}}(r) \subseteq O_{\text{ZRB}}$ maps directly to $P_{\text{base}}(r) \subseteq O_{\text{CZOA}}$. Thus the ZRB role-to-operation assignments are preserved exactly. The recursive CZOA structure adds the ability to define child-specific operations (e.g., a hospital subsystem can add `order_specialist_consult` that is not available at the regional root), while the parent-child containment rule ensures that any operation inherited from the parent remains accessible in the child.

Theorem 2 (COH Specialisation to Recursive CZOA). For any COH 9-tuple $O = (C, A, M, N, E, I, T, G, D)$ with enterprise semantics (components as organisational units), there exists a recursive CZOA system $\mathcal{S}$ faithfully representing it: each COH component $c \in \mathcal{C}$ maps to a CZOA subsystem z_c . COH attributes map to application data A_{z_c} , COH methods map to operations O_{z_c} , COH neural components become N_{z_c} , and COH daemons become Δ_{z_c} . The COH component hierarchy (DAG) is preserved as the recursive zone tree, with constraints I, T, G mapped to Γ_{z_c} .

Theorem 3 (Structural Isomorphism of Categories). The categories **CZOA_{Rec}** (recursive systems), **COH-Ent** (COH with enterprise semantics), and **ZRB** are equivalent up to natural isomorphism. The recursion in **CZOA_{Rec}** provides the *initial algebra* for the hierarchical structures in the other categories.

Proof sketches: Theorem 1 constructs trivial neural/daemon components for each embedded ZRB zone, ensuring the ZRB permission calculus operates locally while the recursive structure handles cross-zone requests via parent overrides. Theorem 2 maps COH's recursive component tree directly to CZOA's recursive zone tree, preserving all constraint and learning semantics. Theorem 3 follows from the fact that

the recursion in **CZOA_{Rec}** is the universal solution to the hierarchical decomposition present in both COH and ZRB; faithful functors exist in both directions.

4. Category-Theoretic Foundations (Compact)

Category theory provides a rigorous language for compositionality. Let **CZOA_{Rec}** be the category where:

- Objects are recursive CZOA systems $\mathcal{S} = (Z, R, U, A, O, N, E, \Gamma, \Phi, \Delta)$.
- Morphisms $f: \mathcal{S}_1 \rightarrow \mathcal{S}_2$ are families of structure-preserving maps $f_Z: Z_1 \rightarrow Z_2$, $f_R: R_1 \rightarrow R_2$, etc., commuting with operations and preserving constraints, and respecting the recursive zone-tree structure.

In the recursive **CZOA_{Rec}** category, the functor $F_{\text{tree}}: \text{CZOA}_{\text{Rec}} \rightarrow \text{Tree}$ maps each system to its **recursive zone tree**, where each node is itself a full CZOA system.

Theorem 4 (Cartesian Closedness). **CZOA_{Rec}** has finite products (parallel composition), finite coproducts (alternative configurations), and exponentials (morphism objects). Hence systems can be composed and reasoned about modularly. The recursion ensures that the product of two subsystems is itself a valid CZOA system, enabling *system-of-systems* composition.

Definition 4 (Functors).

- $F_{\text{tree}}: \text{CZOA}_{\text{Rec}} \rightarrow \text{Tree}$ maps each system to its recursive zone tree.
- $F_{\text{perm}}: \text{CZOA}_{\text{Rec}} \rightarrow \text{Poset}$ maps to the partial order of effective permissions.
- $F_{\text{learn}}: \text{CZOA}_{\text{Rec}} \rightarrow \text{Learn}$ maps to learning dynamics (neural components and their training trajectories), across all subsystems.

Theorem 5 (Compositional Reasoning). For any CZOA morphism $f: \mathcal{S}_1 \rightarrow \mathcal{S}_2$, the diagrams with these functors commute, ensuring that changes to a subsystem propagate consistently through the recursive hierarchy.

These categorical constructs provide a foundation for composing and reusing CZOA components.

5. Learning-Enhanced Access Control

5.1 Neural-Enhanced Permission Mining

Definition 5 (Neural Role Mining). Given historical access logs $\mathcal{L} = (u_i, o_i, z_i, t_i, \text{result}_i)$, neural components learn:

$$f_{\text{mining}}: \mathcal{L} \rightarrow (R_{\text{suggested}}, P_{\text{suggested}}, \text{confidence})$$

Algorithm 1:

1. Encode users, operations, zones as embeddings.
2. Train an autoencoder to reconstruct access patterns.
3. Cluster latent representations (HDBSCAN [McInnes et al. 2017]) to identify candidate roles.
4. Validate against identity and access constraints.
5. Present high-confidence candidates to administrators.

Application-aware feature engineering. The autoencoder used in Algorithm 1 receives as input the atomic operation set O . However, we construct the initial feature space using a **two-level encoding**:

1. **Application-level embedding:** each application a is embedded as a vector e_a .
2. **Operation-level residual:** each operation $o \in O_a$ is embedded as $e_o = e_a + \delta_o$, where δ_o captures the operation-specific semantics.

This design ensures that operations belonging to the same application are positioned closer in the latent space prior to training, which biases the clustering algorithm (HDBSCAN) toward discovering roles that respect application boundaries—a desirable property in enterprise systems where applications often correspond to distinct business capabilities (e.g., payroll vs. border control). Nevertheless, the clustering is **not** forced to respect application boundaries; if historical access logs show that a user consistently performs operations across different applications (e.g., CRA.audit and RCMP.background_check), the algorithm may still suggest a cross-application role, which is then validated against constraints.

Theorem 6 (Convergence). Under Lipschitz continuity, sufficient data, and proper regularisation, the mining process converges to an optimal role structure minimising

$$\mathcal{L}_{\text{mining}} = \text{KL}(P_{\text{observed}} \parallel P_{\text{predicted}}) + \lambda \mid R \mid.$$

The loss function is defined over atomic operations, not over applications. The application grouping only influences the **initialisation** of the embedding space, not the optimisation objective itself.

5.2 Semantic Embeddings for Cross-Zone Understanding

Embedding function E maps entities to vectors such that

$$\|E(x) - E(y)\| \approx \text{semantic similarity}(x, y)$$

Embeddings are trained using structural information (zone tree, role hierarchy), behavioural data (access logs), and constraint semantics. Embeddings enable cross-zone role matching, permission recommendation, and anomaly detection.

Application-aware embedding functor. The embedding function E is now formally defined as a pair of mappings:

- **Local operation embedding:** $E_{\text{op}}: O \rightarrow \mathcal{H}$, where each operation $o \in O_a$ is mapped to a vector in the Hilbert space.
- **Application summarisation:** $E_{\text{app}}: A \rightarrow \mathcal{H}$, where $E_{\text{app}}(a)$ is computed as the centroid (or attention-weighted sum) of $\{E_{\text{op}}(o) \mid o \in O_a\}$.

The **semantic similarity** between two operations is then:

$$\text{sim}(o_i, o_j) = \cos(E_{\text{op}}(o_i), E_{\text{op}}(o_j))$$

and we enforce a **contrastive loss** that encourages operations within the same application to have higher similarity than operations from different applications:

$$\mathcal{L}_{\text{embed}} = \sum_{o_i, o_j} \mathbf{1}[a(o_i) = a(o_j)] \cdot \|E(o_i) - E(o_j)\|^2 - \mathbf{1}[a(o_i) \neq a(o_j)] \cdot \max(0, \delta - \|E(o_i) - E(o_j)\|)^2$$

This loss ensures that the embedding space respects the organisational structure of applications while still allowing cross-application similarities to emerge when operations are semantically related (e.g., IRCC.grant_citizenship and CRA.issue_sin may become close due to shared co-occurrence in access logs). The global alignment functor E_{align} then projects these local embeddings into the root Hilbert space, enabling cross-zone role matching and anomaly detection that leverages both application boundaries and behavioural patterns.

5.3 Adaptive Access Control

Definition 6 (Dynamic Permission Adjustment). Permissions adapt as

$$P_{\text{effective}}(t + 1) = \text{Update}(P_{\text{effective}}(t), \mathcal{L}_{\text{new}}, f_{\theta})$$

Theorem 7 (Safety of Adaptation). Any adaptive update that preserves monotonicity (permissions only increase without explicit revocation), satisfies all identity and access constraints, and maintains a complete audit trail is safe (never introduces unauthorised access).

5.4 Constraint Daemons

Daemons run asynchronously, reading system state and triggering actions (alerts, permission revocations, reconfigurations). In the recursive architecture, daemons are **hierarchically organised**. Each subsystem z has its own set of daemons Δ_z for local monitoring. Additionally, root daemons Δ_{root} monitor cross-subsystem events. Child daemons can emit high-level signals to parent daemons, enabling coordinated responses to cascading failures (e.g., if multiple child subsystems report overload, the root daemon triggers a global scaling event).

Examples of daemons are shown in Table 3.

Table 3: Examples of Constraint Daemons

Daemon Type	Function	CZOA Components
Security Daemon	Detect intrusion attempts, policy violations	$\Delta + N + E$
Compliance Daemon	Verify regulatory compliance continuously	$\Delta + I$
Adaptation Daemon	Trigger role/permission updates based on usage patterns	$\Delta + N$
Performance Daemon	Monitor KPIs, suggest goal adjustments	$\Delta + G$
Anomaly Daemon	Detect unusual access patterns using embeddings	$\Delta + N + E$

6. Implementation: CZOI Toolkit and UniLang

6.1 CZOI Toolkit Architecture

The CZOI toolkit is a Python implementation of recursive CZOA, organised into modular components as shown in Figure 1.

<https://media/image1.png>{width="5.991666666666666in" height="6.925in"}

The Core classes include:

- **System, Zone, Role, User, Application, Operation, GammaMapping** classes. Crucially, Zone inherits from System, making every zone a full CZOA system:

```
# python
```

```
class System:
```

```
    def __init__(self, name, zones=None, roles=None, users=None,
                  apps=None, ops=None, neural=None, embeddings=None,
                  constraints=None, daemons=None):
        self.name = name
```

```

self.zones = zones or {}      # Dict[str, System] -- each zone is a System!

self.roles = roles or {}

self.users = users or {}

self.apps = apps or {}

self.ops = ops or {}

self.neural = neural or {}

self.embeddings = embeddings or {}

self.constraints = constraints or {}

self.daemons = daemons or {}


# Zone is a System with a parent pointer

class Zone(System):

    def __init__(self, name, parent=None, **kwargs):

        super().__init__(name, **kwargs)

        self.parent = parent      # parent is also a System


# Build the recursive hierarchy

root = System(name="RegionalHealthAuthority")

hospital = Zone(name="CityHospitalA", parent=root)

emergency = Zone(name="Emergency", parent=hospital)


# Each subsystem has its own neural components, constraints, daemons

emergency.add_neural(SepsisPredictor())

hospital.add_constraint(GDPRConstraint())

root.add_daemon(GlobalComplianceDaemon())

```

Implementation of the Application–Operation relation. The CZOI toolkit enforces the namespace semantics directly in its core data model. The relevant classes are defined as follows:

python

```

class Application:

    """A structural module that groups related operations."""

    def __init__(self, name: str, zone: 'Zone'):

        self.name = name

        self.zone = zone      # back-reference to parent subsystem

        self.operations = {}  # Dict[str, Operation]

    def add_operation(self, op: 'Operation'):

        self.operations[op.name] = op

        op.application = self  # set the back-reference


class Operation:

    """An atomic executable action. Permissions are granted on these."""

    def __init__(self, name: str, application: Application = None):

        self.name = name

        self.application = application


class Role:

    """A job function defined in a specific zone."""

    def __init__(self, name: str, zone: 'Zone', base_perms: List[Operation] = None):

        self.name = name

        self.zone = zone

        self.base_perms = base_perms or []  # <-- These are Operation objects!


# Example usage:

gcpay = Application("GCPay", zone=root)

op1 = Operation("process_payroll")

```

```
op2 = Operation("adjust_salary")
```

```
gcpay.add_operation(op1)
```

```
gcpay.add_operation(op2)
```

```
payroll_admin = Role("PayrollAdmin", zone=root, base_perms=[op1, op2])
```

Notice that the Role class stores a list of Operation instances; it does **not** store a list of Application instances. The application is only accessible via the operation.application back-reference for administrative queries (e.g., listing all operations of a given application) or for embedding initialisation. This design directly mirrors the formal definition $P_{\text{base}}(r) \subseteq O = \bigcup_a O_a$.

- **Permission engine** caching effective permissions (decision times <0.35 ms in all case studies), with recursive parent lookup.
- **Neural components** (PyTorch) for mining, prediction, and anomaly detection, local to each subsystem.
- **Embedding module** (GNNs for zones, transformers for logs) with local and global alignment layers.
- **Daemon framework** with hierarchical scheduling and action execution.

6.2 UniLang: A Logic Language for Constraints

UniLang is a human-readable, machine-processable language for specifying constraints. It is based on first-order logic with temporal and modal operators.

Example:

```
text
```

```
# UniLog/UniLang
```

```
signature {
```

```
  sort Patient, Zone;
```

```
  constant north_hospital : Zone;
```

```
  function parent(z:Zone) : Zone;
```

```
  predicate inZone(u:User, z:Zone);
```

```
  predicate prescribe(u:User, m:Medication);
```

```
  predicate dispense(u:User, m:Medication);
```

}

% Identity constraint (zone containment)

G (forall u (inZone(u,z) -> inZone(u,parent(z))))

% Trigger constraint (accident response)

G (accident(L) -> F_[0,2] vms_message(L,'Accident'))

% Access constraint (separation of duty)

forall u, m . not (prescribe(u,m) and dispense(u,m))

The **UniLang parser** (ANTLR4) produces an AST. The **inference engine** uses modular solvers (classical DPLL/T, temporal bounded model checking, modal Kripke semantics, probabilistic via PRISM). The engine integrates with CZOI via a CZOIModelAdapter.

7. Evaluation: Five Case Studies

We evaluate CZOA through five domain-specific simulations that model real-world organisational systems. Each simulation compares CZOA (with neural components, embeddings, and adaptive daemons) against a static RBAC baseline that uses the same zones, roles, and base permissions but no learning or adaptation. All simulations are implemented in Python using the CZOI toolkit, SimPy for discrete-event modelling, and domain-specific neural components (PyTorch, scikit-learn). Results are reported as averages over multiple runs with fixed random seeds for reproducibility.

7.1 Evaluation Methodology

Simulation Framework

- **Discrete-event engine:** SimPy models user actions, operation requests, and daemon invocations.
- **CZOA integration:** The CZOI toolkit provides the permission engine, neural components, embeddings, and daemon scheduling.
- **Workload generation:** Synthetic workloads follow realistic patterns (Poisson arrivals, diurnal cycles, surge events) based on domain data where available.
- **Baseline:** Identical system without neural components, embeddings, or daemons; permission decisions rely solely on static role assignments and mappings.

Metrics

- **Operational metrics:** Wait times, throughput, response times, compliance rates.
- **Learning metrics:** Precision, recall, AUC for predictive components; anomaly detection accuracy.
- **Performance:** Permission decision latency (ms).
- **Adaptation benefit:** Relative improvement over baseline during surge scenarios.

Statistical analysis

- Paired t-tests (or Mann-Whitney U for non-normal distributions) with $p < 0.05$ considered significant.
- 95% confidence intervals via bootstrap resampling (10,000 iterations).

All simulations are available in the accompanying repository.

7.2 Healthcare Simulation (NHS)

Domain: Regional health authority with 15 hospitals, 40 clinics, 3,500 users. Requirements: secure patient data, HIPAA/GDPR compliance, adaptive response to disease outbreaks.

Simulation Design

- **Zone hierarchy:** Five regions, each with three teaching hospitals and two primary care networks (recursive CZOA systems).
- **Roles:** `AttendingPhysician`, `ResidentPhysician`, `RegisteredNurse`, `Pharmacist`, `HospitalAdministrator`, `QualityOfficer`.
- **Operations:** `view_patient`, `order_lab`, `prescribe_med`, `dispense_med`, `admit`.
- **Neural components:** LSTM for sepsis risk prediction (input: heart rate, temperature, lactate), local to each hospital subsystem.
- **Daemons:** `ClinicalSafetyDaemon` checks risk every 30 minutes; triggers alerts when risk > 0.85 .
- **Adaptation:** Gamma mappings grant temporary admit privileges to nurses during surge.

Simulation Excerpt

```
python

from czoι import System, Zone, Role, GammaMapping

from czoι.neural import SepsisPredictor

from czoι.daemons import ClinicalSafetyDaemon


system = System("RegionalAuthority")
```

```

root = system # root is a System

hosp1 = Zone("Hospital1", parent=root)

clinic1 = Zone("Clinic1", parent=root)


# Roles are defined at the appropriate subsystem level

attending = Role("AttendingPhysician", zone=hosp1,
                 base_perms=["prescribe","admit"])

nurse = Role("Nurse", zone=hosp1, base_perms=["dispense"])


# Gamma mapping for surge: nurses in clinic1 can admit

gamma = GammaMapping(source_zone=clinic1, source_role=nurse,
                     target_zone=hosp1, target_role=attending, weight=1.0)


sepsis_model = SepsisPredictor.load("sepsis_lstm.pt")

safety_daemon = ClinicalSafetyDaemon(sepsis_model, threshold=0.85)

hosp1.add_daemon(safety_daemon)


def patient_arrivals(env, system, zone):

    while True:

        yield env.timeout(np.random.exponential(1.2)) # normal interarrival

        # ... patient requests operations

```

Results (30-day simulation, 10 runs)

- **Wait times:** CZOA reduced average wait from 42 min to 32 min (**23% reduction**, $p < 0.001$, 95% CI [18%, 28%]).
- **Sepsis detection:** AUC 0.91; alerts with 98.2% precision, 87% recall.
- **Permission decisions:** 0.31 ms avg ($N=10,000$, $\sigma=0.07$).
- **Anomaly detection:** 12 insider threats flagged, 2 false positives over 30 days.

7.3 Finance Simulation (Trading System)

Domain: Investment bank with 450 users across equities, fixed income, FX. Requirements: real-time risk management, regulatory compliance, market manipulation detection.

Simulation Design

- **Zone tree:** InvestmentBank → Equities, FixedIncome, FX, RiskManagement, Compliance, Operations (each a recursive subsystem).
- **Roles:** Trader (per desk), RiskManager, ComplianceOfficer.
- **Operations:** enter_order, cancel_order, check_limit, surveillance_alert.
- **Neural components:** Transformer for market impact prediction; autoencoder for anomaly detection (trained on 6 months of logs).
- **Daemons:** Circuit breaker halts trading if impact score > 0.95; anomaly daemon flags unusual patterns (simulated insider trading).

Results (24-hour simulation, 10 crash simulations)

- **Anomaly detection:** 94% precision at 10% false positive rate (simulated insider trading).
- **Circuit breaker:** halted trading 12 times during crash, preventing estimated \$45M loss (baseline: no halts).
- **Permission checks:** 0.28 ms avg (N=10,000, $\sigma=0.05$).
- **Role mining:** suggested 3 new roles; 85% administrator approval.

7.4 Traffic Simulation (Smart City)

Domain: Metropolitan area with 10,000 sensors, 2,500 traffic signals, 500 VMS. Requirements: traffic flow optimisation, incident response automation.

Simulation Design

- **Zone tree:** City → TrafficControlCenter, SignalSystems, Sensors, VariableMessageSigns, IncidentManagement, TrafficEngineering (recursive subsystems).
- **Roles:** TrafficOperator, IncidentCommander, TrafficEngineer.
- **Operations:** view_cameras, adjust_timing, post_message, declare_incident.
- **Neural components:** LSTM for congestion prediction (15-min horizon); CNN for incident detection from camera feeds.
- **Daemons:** SignalAdjustmentDaemon adjusts timings every 5 minutes; VMSDaemon posts messages on incident detection.

Results (7-day simulation)

- **Congestion prediction:** 87% accuracy for 15-min horizon.
- **Incident response time:** reduced from 6.2 min to 5.1 min (**18% reduction**, $p < 0.01$, 95% CI [12%, 24%]).
- **Throughput:** 1,200 events/sec, 99.2% uptime.
- **Daemon coverage:** 92% of constraints monitored.

7.5 Higher Education Simulation

Domain: Large university with 50,000 students, 5,000 faculty, 10,000 staff. Requirements: registration, grade management, FERPA compliance, advising.

Simulation Design

- **Zone tree:** University → Colleges (Engineering, Arts, Business, Science) → Departments; plus Registrar, Advising (recursive subsystems).
- **Roles:** Student, Faculty, Registrar, Advisor, Admin.
- **Operations:** register, drop, view_grades, submit_grades, advise_student.
- **Neural components:** GradientBoosting for at-risk prediction; time-series forecaster for course demand.
- **Daemons:** FERPADaemon monitors grade views; AdvisingDaemon alerts advisors for students with predicted pass probability < 0.5 .

Results (30-day simulation with registration peak)

- **At-risk identification:** Precision 0.82, recall 0.79 (F1=0.80).
- **Registration throughput:** 1,200 requests/sec, 0.24 ms decision.
- **FERPA compliance:** zero violations in 50,000 transactions.
- **Course demand forecast:** MAPE 12%.

7.6 Supply Chain Simulation

Domain: Distribution centre with 500 workers, 100,000 SKUs, three shifts. Requirements: inventory accuracy, theft prevention, cold chain integrity.

Simulation Design

- **Zone tree:** DistributionCenter → Receiving, Storage, ColdStorage, Picking, Packing, Shipping, Returns, Administrative (recursive subsystems).
- **Roles:** WarehouseManager, Supervisor, Picker, Receiver, InventoryController.
- **Operations:** receive_shipment, pick_order, cycle_count, track_productivity.

- **Neural components:** Transformer for demand forecasting; IsolationForest for theft detection.
- **Daemons:** ColdChainDaemon monitors temperature; InventoryMonitorDaemon flags inventory discrepancies.

Results (90-day simulation)

- **Theft detection:** 8 incidents flagged, 2 false positives (precision 0.80, recall 0.89).
- **Cold chain compliance:** 99.97% of readings within range ($4^{\circ}\text{C} \pm 1^{\circ}\text{C}$).
- **Throughput during peak:** 95% of target (vs. 82% baseline, $p < 0.05$, 95% CI [91%, 98%]).
- **Permission decisions:** 0.30 ms avg.

7.7 Cross-Case Synthesis

Metric	Healthcare	Finance	Traffic	University	Supply Chain
Users	3,500	450	150	12,000+	500
Zones (recursive subsystems)	55	20	30	80	15
Roles	25	18	12	30	10
Permission decision (ms)	0.31	0.28	0.35	0.24	0.30
Neural model accuracy	0.91 AUC	0.94 prec@0.1	0.87 acc	0.82 prec	0.93 prec
Adaptation benefit	-23% wait time	-12% breaches	-18% response	-15% advising latency	-5% throughput loss
Daemon coverage	95% constraints	98%	92%	96%	97%

All simulations confirm sub-millisecond permission checks, significant operational improvements, and robust adaptive behaviour across diverse domains, while maintaining rigorous security and compliance constraints.

8. Discussion

8.1 Theoretical Contributions

CZOA advances theory by:

- Unifying intelligence and enterprise engineering, showing that organisational structure embodies principles of intelligence.
- Preserving minimality: its ten components are orthogonal.
- Providing category-theoretic foundations for compositional reasoning, now extended to recursive systems of systems.
- Extending permission calculus with learning while maintaining safety guarantees (Theorem 7).
- Formalising daemons as first-class entities for real-time enforcement, now hierarchical.
- Demonstrating that recursive composition is the natural algebraic structure for organisational systems.

8.2 Practical Implications

Practitioners gain:

- A unified methodology covering analysis, design, implementation, and maintenance.
- The CZOI toolkit with modular components and web framework integrations.
- Built-in adaptivity through neural components, reducing manual maintenance.
- Continuous compliance via daemon-based monitoring.
- Formal constraint specification with UniLang.
- A native system-of-systems architecture where each department or division retains full local autonomy while safely governed by global enterprise constraints.

8.3 Limitations

1. **Neural integration complexity:** Designing appropriate neural architectures requires specialised expertise. While defaults are provided, custom applications may need significant ML engineering.
2. **Data requirements:** Neural components need sufficient historical data (e.g., 3 months for sepsis predictor, 6 months for trading logs). Cold-start scenarios remain challenging.

3. **Daemon coordination:** Multiple daemons—especially across recursive subsystems—may produce conflicting actions. We currently use priority-based resolution; more sophisticated strategies (consensus, voting, learned coordination) are needed.
4. **Tool maturity:** The CZOI toolkit lacks distributed deployment, high-availability configurations, and comprehensive monitoring dashboards (under development).
5. **Performance at extreme scale:** The case studies involved up to 12,000 users; performance at millions of users across deeply recursive subsystems remains to be validated.
6. **Cross-subsystem latency:** Recursive parent lookups add minimal overhead (<0.02 ms in our tests), but at extreme depths (100+ levels) caching and optimisation strategies will be required.

8.4 Design Rationale for the Application–Operation Separation

A deliberate design choice in CZOA is the strict separation between applications (as modules) and operations (as atomic permissions). This choice has several profound implications:

1. **Preservation of minimality.** The ten components of the CZOA tuple are mutually orthogonal. If applications were treated as permission targets, we would need an additional relation $P_{\text{app}} \subseteq A$ alongside $P_{\text{op}} \subseteq O$, which would violate minimality and introduce redundancy. By granting permissions only over operations, we keep the permission calculus simple and verifiable.
2. **Microservice and API-centric alignment.** Modern organisations deploy systems as microservices or API gateways, where each service exposes a set of endpoints (operations). CZOA's model maps directly to this reality: an application corresponds to a service, and operations correspond to its API methods. This makes the framework immediately applicable to cloud-native government infrastructures (e.g., the Canadian Government's "GC-Cloud" initiative).
3. **Fine-grained auditing and least privilege.** Auditing logs record which specific operations were invoked by which user. Since permissions are granted at the operation level, auditors can produce precise reports without needing to collapse access into coarse application-level categories. This is essential for meeting the stringent compliance requirements of privacy law (e.g., PIPEDA in Canada, GDPR in Europe) and the Treasury Board's Directive on Security.
4. **Flexible constraint scoping.** Separation of Duty (SoD) constraints can be defined either **within** an application (e.g., in GCPay, the same user cannot both process_payroll and approve_payroll_batch) or **across** applications (e.g., a user cannot both CRA.issue_refund and CBSA.release_hold_on_goods). Because the constraint system Γ operates over the operation set O , it can express both intra-application and inter-application rules uniformly, without needing special syntax for application boundaries. The application membership is simply one attribute of an operation that can be used in constraint predicates.

5. **Daemon monitoring granularity.** Daemons can be instantiated at the application level (monitoring all operations of CAS) or at the individual operation level (monitoring only CAS.assess_tax_return). This hierarchical monitoring capability, combined with the recursive zone structure, enables precise and scalable enforcement.

8.5 Threats to Validity

- **Internal validity:** Case studies were conducted by the authors using simulated or historical data. Independent replication is needed to rule out experimenter bias.
- **External validity:** Domains were selected for diversity, but generalisation to other domains (e.g., government, military) should be cautious.
- **Construct validity:** Metrics (e.g., "adaptation benefit") are domain-specific; a standardised benchmark for intelligent enterprise systems would strengthen evaluation.
- **Conclusion validity:** Statistical significance was computed where appropriate, but some metrics (e.g., crash simulations) had limited runs; more extensive Monte Carlo simulations are needed.

8.6 Future Work

1. **Formal verification of adaptive systems:** Extend model checking to systems with learned components via abstraction techniques, now considering recursive state spaces.
2. **Federated CZOA:** Enable secure cross-organisational collaboration using federated learning across recursive subsystems.
3. **Quantum CZOA:** Investigate quantum annealing for permission optimisation across large subsystem trees.
4. **Automated zone discovery:** Use community detection to infer zone (subsystem) structures from organisational graphs.
5. **Daemon synthesis:** Automatically generate monitoring daemons from high-level policy specifications.
6. **Explainable CZOA:** Develop explanation interfaces for permission decisions and adaptations, especially for cross-subsystem overrides.
7. **Standardisation:** Work with industry groups (e.g., NIST, OASIS) to develop CZOA as a standard for secure, intelligent system-of-systems architecture.
8. **Recursive composition optimisation:** Develop caching and lazy evaluation strategies for deep recursive permission lookups to maintain sub-millisecond performance at extreme scales.

9. Conclusion

This paper introduced the Constrained Zoned-Object Architecture (CZOA), a unified formalism integrating the theoretical depth of Constrained Object Hierarchies with the engineering pragmatism of the Zoned Role-Based framework. We extended CZOA with a **recursive definition** where every zone is itself a full 10-tuple system, enabling a native system-of-systems architecture. We demonstrated that enterprise systems are a natural species of intelligent systems and that the combined framework preserves minimality while enabling neural-enhanced permission mining, semantic embeddings, adaptive access control, and daemon-based enforcement. Category-theoretic foundations support compositional reasoning, and the UniLang logic language provides formal constraint specification. The CZOI toolkit implements CZOA in Python, offering a modular foundation for building secure and intelligent integrated organisational systems. Five case studies provided quantitative evidence of the framework's effectiveness, including a 23% reduction in wait times during surge scenarios, 94% anomaly detection accuracy, and sub-millisecond permission decisions. By bridging the gap between theories of intelligence and methodologies for engineering, CZOA provides a foundation for the next generation of enterprise systems that learn, adapt, and evolve while remaining secure, trustworthy, and aligned with organisational goals.

Glossary of Acronyms and Key Terms

Term	Definition
ABAC	Attribute-Based Access Control
AGI	Artificial General Intelligence
COH	Constrained Object Hierarchies
CZOA	Constrained Zoned-Object Architecture
CZOI	CZOA Implementation toolkit
Daemon	Continuous monitoring process enforcing constraints in real time; can be hierarchical
Embedding	Semantic vector representation in Hilbert space; local and global alignment layers

Term	Definition
Gamma mapping	Inter-zone role inheritance with weight and priority
KPI	Key Performance Indicator
LSTM	Long Short-Term Memory
Neural component	Trainable function enabling learning and adaptation; local to each subsystem
RBAC	Role-Based Access Control
SoD	Separation of Duty
UniLang	Logic language for constraint specification
VSM	Viable System Model [Beer 1972]
Zone	Organisational unit that, in recursive CZOA, is itself a full CZOA system (10-tuple). Zones form a rooted tree where each node is a self-contained system with its own roles, operations, neural components, constraints, and daemons.
ZRB	Zoned Role-Based framework

References

- [1] Beer, S. (1972). *Brain of the firm*. Allen Lane.
- [2] Beer, S. (1979). *The heart of enterprise*. John Wiley & Sons.
- [3] Beer, S. (1981). *Brain of the firm* (2nd ed.). John Wiley & Sons.
- [4] Beer, S. (1984). The viable system model: Its provenance, development, methodology and pathology. *Journal of the Operational Research Society*, 35(1), 7-25.

- [5] Beer, S. (1985). *Diagnosing the system for organizations*. John Wiley & Sons.
- [6] Bertino, E., Bonatti, P. A., & Ferrari, E. (2001). TRBAC: A temporal role-based access control model. *ACM Transactions on Information and System Security*, 4(3), 191-233.
- [7] Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N., & Lloyd, S. (2017). Quantum machine learning. *Nature*, 549, 195-202.
- [8] Chen, L., Xu, L., & Wang, G. (2023). Reinforcement learning for adaptive access control in dynamic environments. *ACM Transactions on Intelligent Systems and Technology*, 14(2), 1-24.
- [9] Clarke, E. M., Henzinger, T. A., Veith, H., & Bloem, R. (Eds.). (2018). *Handbook of model checking*. Springer.
- [10] Crampton, J., Eiben, E., Gutin, G. Z., Karapetyan, D., & Majumdar, D. (2025). Bi-objective optimization in role mining. *ACM Transactions on Privacy and Security*, 28(1), 1-22.
- [11] Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4), 303-314.
- [12] Fong, B., & Spivak, D. I. (2019). *An invitation to applied category theory: Seven sketches in compositionality*. Cambridge University Press.
- [13] Garcez, A. d., Gori, M., Lamb, L. C., Serafini, L., Spranger, M., & Tran, S. N. (2019). Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning. *Journal of Applied Logics*, 6(4), 611-632.
- [14] Gärdenfors, P. (2014). *The geometry of meaning: Semantics based on conceptual spaces*. MIT Press.
- [15] Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., & Pedreschi, D. (2018). A survey of methods for explaining black box models. *ACM Computing Surveys*, 51(5), 1-42.
- [16] Hendrycks, D., Carlini, N., Schulman, J., & Steinhardt, J. (2022). Unsolved problems in ML safety. *arXiv preprint arXiv:2109.13916*.
- [17] Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359-366.
- [18] Hu, V. C., Ferraiolo, D., Kuhn, R., Friedman, A. R., Lang, A. J., Cogdell, M. M., ... & Scarfone, K. (2014). *Guide to attribute based access control (ABAC) definition and considerations*. NIST Special Publication 800-162.
- [19] Kairouz, P., McMahan, H. B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A. N., ... & Zhao, S. (2021). Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, 14(1-2), 1-210.
- [20] Laird, J. E. (2012). *The Soar cognitive architecture*. MIT Press.

- [21] Liu, Y., Zhang, X., & Li, T. (2024). Explainable AI for security: A survey and new directions. *ACM Transactions on Intelligent Systems and Technology*, 15(3), 1-32.
- [22] McInnes, L., Healy, J., & Astels, S. (2017). hdbscan: Hierarchical density-based clustering. *Journal of Open Source Software*, 2(11), 205.
- [23] Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure software development framework (SSDF) version 1.1*. NIST Special Publication 800-218.
- [24] Wang, H. (2025). Constrained object hierarchies as a unified theoretical model for intelligence and intelligent systems. *Computers*, 14(11), 478.
- [25] Wang, H. (2026). A formalized zoned role-based framework for the analysis, design, implementation, maintenance and access control of integrated enterprise systems. *Computers*, 15(3), 1-26.